App Dev Project Report

1. Student Details

Name: Piyush Maharana

Roll Number: 24f2006489

Email: 24f20064f89@ds.study.iitm.ac.in

About Me: I am a student at IIT Madras Bs Degree program with a little interest in

Web application development. I enjoy learning about building application that combines analytics and user experience. I prefer myself calling 'jack of all trades but master of none'. But if we dig deeper into this, it says "Jack of all trades but master of none, but oftentimes better than master of one.

---

2. Project Details

- **Project Title: Hospital Management System (HMS)**

- **Problem Statement:**

  To design and build a multi-user, role-based web application that allows a hospital to efficiently manage patients, doctors, and appointments. The primary challenge is to replace manual or disconnected systems, prevent scheduling conflicts, and provide secure, instant access to patient history.

- **Approach:**

The application was built using Flask as a backend framework with a modular structure. It provides three secure, role-based dashboards (Admin, Doctor, Patient). Admins have full CRUD control over users. Doctors manage their 7-day repeating availability and add treatment records. Patients can browse doctors and book appointments based on this real-time availability. The system is rendered entirely on the server-side, with charts generated by Matplotlib.

---

3. AI/LLM Declaration

I used Google's Gemini as an extensive pair-programming partner throughout the project. The AI was used to:

- Refactor complex logic, such as redesigning the booking system from individual "slots" to a 7-day repeating schedule (`DoctorAvailability` model).
- Debug and resolve errors by analyzing traceback logs (e.g., `sqlite3.OperationalError`, `UnboundLocalError`).
- Generate the CSS for the professional two-column login/register pages based on a reference image.
- Generate the Matplotlib code for the server-side chart.

The extent of AI/LLM usage is high, as it was involved in debugging and styling phases. All core application logic, database design decisions, and final integrations were directed, tested, and validated by me.

Component / Tool	Purpose / Role in Project	MAD-1 Usage %

Flask (App)	Backend logic, routing, request handling	25%
SQLite + SQLAlchemy	Data modeling, relationships, persistence	15%
HTML + Jinja2	Server-side template rendering	20%
Bootstrap / CSS	Styling, layout, (custom `login_style.css`, `landing.css`)	10%
JavaScript / AJAX	N/A (We intentionally avoided this)	0%
Authentication (Login/Auth)	Login/logout, role-based access control (RBAC)	10%
Admin CRUD Logic	Manage Doctors and Patients (Add, Edit, Blacklist)	5%
User Logic (Booking)	7-day availability, slot generation, booking, history	10%
Validation (Client + Server)	Server-side checks (e.g., duplicate slots, past dates)	3%
Charts / Data Visualization	Matplotlib server-side chart for Admin dashboard	2%
Testing / Debugging	N/A (Process, not a weighted component)	0%
APIs / External Integration	N/A (Skipped this optional milestone)	0%

Category	**MAD-1 Total %**	**Notes**

Backend (Flask + DB)	40%	Core of the application (logic and models)
Frontend (HTML/Jinja)	20%	All pages were server-side rendered
Styling/UI	10%	Includes custom login/landing pages
Auth + User Logic	25%	Includes all 3 roles (Admin, Doctor, Patient)
Extras (Charts/API/etc)	5%	Includes Matplotlib chart and Validation

4.Technologies and Framework Used

Technology / Library	Purpose
Flask	Core backend web framework
SQLAlchemy	Object Relational Mapper for the SQLite database
Jinja2	Template engine for rendering dynamic HTML pages
Bootstrap 5	Primary frontend framework for styling and layout
Matplotlib	Server-side generation of chart images for the admin dashboard

Flask-Login	User authentication and session management
Flask-Bcrypt	Secure password hashing and verification
SQLite	Lightweight local database for storing all application data

---

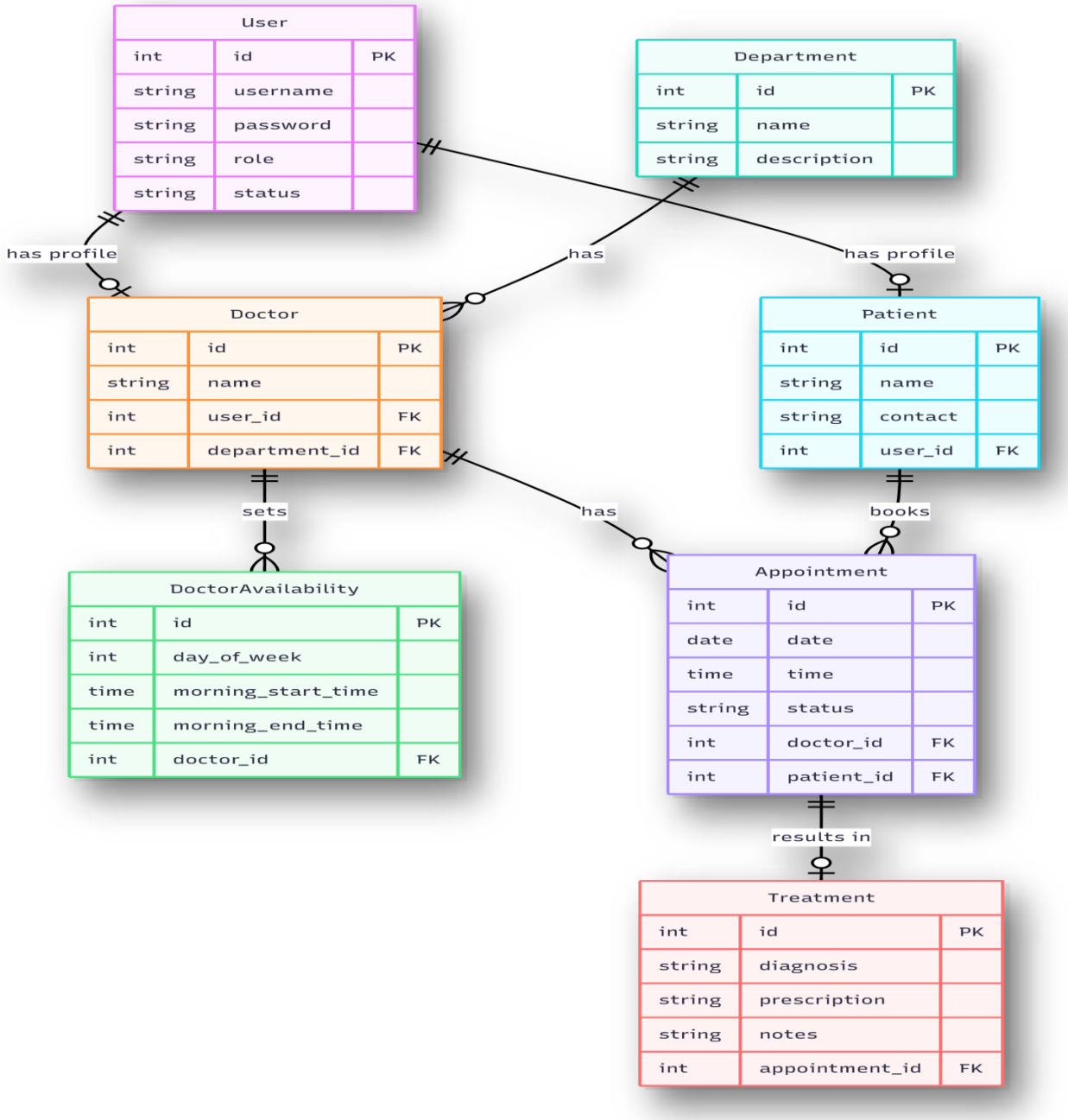
5. Database Schema / ER Diagram

Tables:

- User: Stores login details for all roles (id, username, password, role, status).
- Department: Stores medical departments (id, name, description).
- Doctor: Stores doctor-specific profiles (id, name, contact, user_id, department_id, etc.).
- Patient: Stores patient-specific profiles (id, name, contact, user_id).
- DoctorAvailability: Stores the doctor's 7-day repeating schedule (id, doctor_id, day_of_week, shift times).
- Appointment: Logs all appointments, linking doctors and patients (id, date, time, status, doctor_id, patient_id).
- Treatment: Stores details of a completed appointment (id, diagnosis, prescription, notes, appointment_id).

Relationships:

- One-to-One → `User` → `Doctor`
- One-to-One → `User` → `Patient`
- One-to-Many → `Department` → `Doctor`
- One-to-Many → `Doctor` → `DoctorAvailability` (one for each day)
- One-to-Many → `Doctor` → `Appointment`
- One-to-Many → `Patient` → `Appointment`
- One-to-One → `Appointment` → `Treatment`



6.API Resource and Endpoints

This project was built as a server-side rendered Flask application. As per our project plan, the optional API milestone was not implemented.

All data is handled through secure Flask routes, form submissions, and rendered via Jinja2 templates.

---

7. Architecture and Features

Architecture Overview:

- **`app.py`:** Main Flask application entry point. Contains all routes, logic, and `flask_login` configuration.
- **`models.py`:** Defines all database models using SQLAlchemy.
- **`init_db.py`:** A utility script to programmatically create the database, tables, and a predefined admin user.
- **`/templates`:** Contains all Jinja2 HTML files.
  - **`layout.html`:** The master template used by all authenticated pages for a consistent look.
  - **Dashboard templates:** `admin_dashboard.html`, `doctor_dashboard.html`, `patient_dashboard.html`.
  - **And many others...**
- **`/static`:** Contains CSS and image files.
  - **`css/style.css`:** Custom styling for the main application.
  - **`css/login_style.css`:** Custom styling for the two-column login/register pages.

Implemented Features:

- **Role-Based Access Control (RBAC):** Secure dashboards and routes for Admin, Doctor, and Patient roles.
- **Full Admin CRUD:** Admins can create, read, update, delete, and blacklist/activate both Doctors and Patients.
- **Doctor Availability:** Doctors can set their 7-day repeating weekly schedule (morning/evening shifts).

- **Patient Booking:** Patients see a 7-day list of available 30-min slots based on the doctor's schedule and existing appointments.
- **Conflict Prevention:** The system prevents double-booking of a time slot.
- **Appointment Management:** Doctors can mark appointments as "Completed" and add `Treatment` records.
- **History Tracking:** All roles (Admin, Doctor, Patient) can view a patient's past medical history.
- **Professional UI:** A consistent, responsive UI built with Bootstrap 5 and a custom two-column login page.
- **Server-Side Charting:** The Admin dashboard features a Matplotlib-generated bar chart (no JavaScript).

8. Video Presentation

https://drive.google.com/file/d/1QCN7ItyQMcIx5XTxDnVnIlAV5rvF1JmT/view?usp=sharing